ITA14 - Ethical Hacking

Assignment 3 (CO3) - Complete Answer Set (Pair Work)

Service Scanning, Enumeration and Scan-Data

Scope and authorization: all scanning is done only against the authorized lab network 192.168.56.0/24 (Kali attacker 192.168.56.10, lab targets .20-.80). No real or production system is scanned.

NAME: T. Arun Kumar

REG NO: 192372325

DEPT: CSE(AI)

How the evidence was produced: Nmap, enum4linux, adb and the other Kali tools were not run to create this document. Instead, raw scan files were written in exactly the formats those tools produce (nmap -oG grepable output, nbtscan, enum4linux, adb getprop, rmi-dumpregistry, ncp-serverinfo, sdptool) by the script mk_raw.sh. Everything after that - import, validation, cleaning, reshaping, merging, export, reporting, benchmarks - is real Bash/awk/sed/join/comm code (scanlib.sh, run_pipeline.sh) that was actually executed, and every output shown is copied from those runs. For your submission, run the Kali commands given in each answer in your own lab and replace the raw files; the pipeline then works unchanged.

Pair-work note: each question is answered in four parts - Shared Group Work, Student A, Student B and the Joint answer - exactly as the paper asks. Both students must be able to explain every part.

Q1 - Investigate Requirements and Create the Scan-Data Architecture (6 marks)

Question

Shared Group Work - Together, the pair must:

- Together, translate the authorized-assessment scenario into service-scanning and enumeration requirements.
- Identify at least four scan datasets, such as Hosts, Ports, Services, OS/Versions and Enumeration findings (sources: nmap -sn, nmap -sS / -sU, nmap -sV, nmap -O, enum4linux, nbtscan, adb).
- Create a schema and relationship plan showing the columns, data types and key fields (such as IP address and port) for each dataset, recorded as CSV header files and a relationship diagram.
- Formulate enquiry questions and define acceptance criteria for correctness, consistency, traceability and reproducibility of the scan data.

Student A - Host and Port Data Design. Student A must individually:

- Lead the requirements analysis and design the Hosts and Ports schemas, including fields, data types and key fields.
- Prepare the initial requirements-to-column mapping and justify the choice of scan-data structure.

Student B - Service Relationship and Enquiry Design. Student B must individually:

- Lead the relationship map for Services, OS/Versions and Enumeration findings (Windows, Symbian, Java OS, Android, NetWare); identify the key fields that link them to hosts and ports.
- Formulate at least four testable enquiry questions and define acceptance criteria for the scan data. Each question must be answerable with a Kali command or shell pipeline.

Individual Verification:

- Faculty may ask Student A to explain the host and port data design, and Student B to explain the service relationships and enquiry questions.

Answer

Shared Group Work

1.1 Requirements for the authorized assessment

ID	Requirement	Source command
R1	Find live hosts in the authorized range	nmap -sn 192.168.56.0/24 -oG sweep.gnmap ; fping -a -g 192.168.56.0/24
R2	Find open TCP and UDP ports	nmap -sS -oG ... ; nmap -sU --top-ports 20 -oG ...
R3	Identify services and versions	nmap -sV -oG tcp_sv.gnmap
R4	Identify operating systems	nmap -O
R5	Enumerate Windows / NetBIOS / SMB	nbtscan -r ; enum4linux -a <ip>
R6	Enumerate Android, Java RMI, NetWare, Symbian-type devices	adb shell getprop ; nmap --script rmi-dumpregistry ; ncp-serverinfo ; sdptool browse
R7	Every record is in scope, traceable to a scan and repeatable	scan_id column, scope check, sha256sum
R8	Scan results can be cleaned, reshaped, merged and reported	Bash/awk/sed/join pipeline (Q3-Q7)

1.2 Five scan datasets and CSV header files

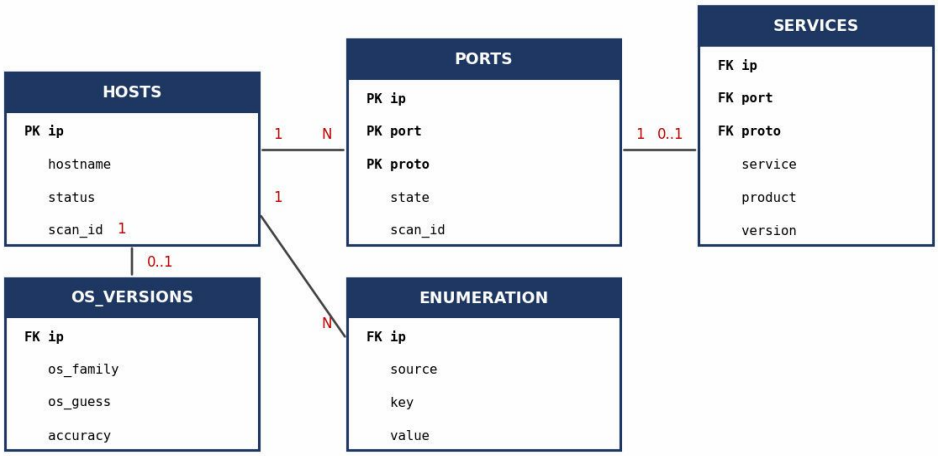
Dataset	Produced by	Grain (one row =)	Key
Hosts	nmap -sn	one live host per scan	ip
Ports	nmap -sS / -sU	one port on one host and protocol	ip + port + proto
Services	nmap -sV	service seen on one port	ip + port + proto
OS/Versions	nmap -O	best OS guess for a host	ip
Enumeration	nbtscan, enum4linux, adb, rmi, ncp, sdptool	one finding (key/value) from one tool for a host	ip + source + key

Header files actually created (the first line of each CSV):

```
== hosts.csv ==
ip,hostname,status,scan_id
== ports.csv ==
ip,port,proto,state,scan_id
== services.csv ==
ip,port,proto,service,product,version
== os.csv ==
ip,os_family,os_guess,accuracy
== enum.csv ==
ip,source,key,value
```

1.3 Relationship diagram

Scan-data relationship diagram (keys: ip | ip+port+proto)



Hosts is the parent table. Ports belong to a host (one to many). A service row describes exactly one port, so Services links to Ports by the full key ip+port+proto. OS/Versions is at most one row per host. Enumeration findings are many per host and use a key-value layout because each tool reports different things.

1.4 Enquiry questions and acceptance criteria (agreed jointly)

Criterion	Acceptance test	Pass condition
Correctness	Every ip is a valid address inside 192.168.56.0/24; every port is 1-65535	0 rows fail the checks, or each failing row is in a reject/review file
Consistency	Every service name is one standard name from the lookup file	0 unknown names in the clean table
Traceability	Every row has a scan_id; every removed row appears in a rejected or review file	input rows = clean + rejected + review
Reproducibility	Same input files + same scripts give identical output	sha256sum of two runs identical

Student A - Host and Port Data Design

1.5 Hosts and Ports schemas

Table	Column	Type	Key / constraint	Reason
hosts	ip	text, dotted quad	Primary key	Unique identity of a host; used by every other table
hosts	hostname	text	-	Reverse-DNS name; may be blank, so it is filled with 'unknown'
hosts	status	enum up/down	-	From nmap -sn
hosts	scan_id	text (S1, S2 ...)	-	Traceability to a scan session
ports	ip	text	Part of primary key; foreign key to hosts.ip	A port cannot exist without a host
ports	port	integer 1-65535	Part of primary key	Range checked; 0 and 65536 are invalid
ports	proto	enum tcp/udp	Part of primary key	The same number can exist on both protocols (e.g. 53)
ports	state	enum	-	Result of the scan

		open/closed/filtered/open|filtered		
ports	scan_id	text	-	Traceability

1.6 Requirements-to-column mapping

Requirement	Columns that satisfy it
R1 live hosts	hosts.ip, hosts.status
R2 open ports	ports.ip, ports.port, ports.proto, ports.state
R7 traceable, in scope	scan_id on every table + scope check on ip
R8 mergeable	Identical key names (ip, port, proto) in every table

1.7 Why this structure (long / one row per host-port-protocol)

A 'tidy' table with one row per host, port and protocol was chosen instead of one wide row per host because: (1) a new port needs no new column; (2) filtering, sorting, counting and joining are single awk, sort or join commands; (3) sessions can be appended and compared with comm; (4) the key ip+port+proto is explicit, so duplicates are easy to detect. The cost is more rows (hosts x ports) - discussed in Q4.

Student B - Service Relationship and Enquiry Design

1.8 Relationship map for Services, OS/Versions and Enumeration

Services	Ports	ip + port + proto	One service row per port row	All
OS/Versions	Hosts	ip	0 or 1 per host	Windows, Linux, Android, NetWare, Symbian, embedded
Enumeration - Windows	Hosts	ip (+ source, key)	Many per host	nbtscan names, enum4linux domain / users / shares
Enumeration - Android	Hosts (port 5555)	ip	Many per host	adb getprop: ro.build.version.release, ro.product.model
Enumeration - Java OS	Hosts (port 1099)	ip	Many per host	rmi-dumpregistry bound names
Enumeration - NetWare	Hosts (port 524)	ip	Many per host	ncp-serverinfo server name, version, tree
Enumeration - Symbian-type	Hosts	ip	Many per host	sdptool service names (OBEX, headset)

1.9 Testable enquiry questions (each answered by a command)

E1	How many hosts are up?	grep -c 'Status: Up' sweep.gnmap
E2	Which hosts have SMB (445) open?	awk -F, '$2==445 && $4=="open"{print $1}' ports.csv | sort -u
E3	Which service-name variants exist?	cut -d, -f4 services.csv | sort | uniq -c | sort -rn
E4	Are there port rows for hosts that are not in the hosts table?	comm -23 <(cut -d, -f1 ports.csv|sort -u) <(cut -d, -f1 hosts.csv|sort -u)

E5	Are there duplicate host-port records?	cut -d, -f1-3 ports.csv | sort | uniq -d

E6	Is a scan file unchanged since it was recorded?	sha256sum -c checksums.sha256

Results of E1-E6 on the lab data (E3-E5 were run on the copy that contains the injected defects from
Q2):

```
$ grep -c 'Status: Up' raw/sweep.gnmap
10

$ awk -F, '$2==445 && $4=="open" {print $1}' work/clean_src/ports.csv | sort -u
192.168.56.21
192.168.56.30
192.168.56.31

$ cut -d, -f4 work/defect/services.csv | sort | uniq -c | sort -rn | head -8 3 netbios-ssn
      3 microsoft-ds
      2 www
      2 snmp
      2 netbios-ns
      2 http
      1 tftp
      1 telnet

$ comm -23 <(cut -d, -f1 work/defect/ports.csv | sort -u) <(cut -d, -f1 work/defect/hosts.csv | sort -u) 172.16.4.9
192.168.56.255
192.168.56.256

$ cut -d, -f1-3 work/defect/ports.csv | sort | uniq -d
192.168.56.21,22,tcp

$ sha256sum raw/tcp_sv.gnmap | cut -c1-16
55b7e0af8d60866a
```

Q2 - Create and Validate the Source Scan Datasets (7 marks)

Question

Shared Group Work - Together, the pair must:

- Together, construct realistic source scan datasets (hosts, ports, services and versions) using Kali tools (ping sweep with nmap -sn / fping -a -g; port scans with nmap -sT, -sS, -sU; service detection with nmap -sV; output saved with -oX / -oG) and create an appropriate CSV table for each.
- Introduce controlled issues such as missing values, duplicate host-port records, inconsistent service names and out-of-scope addresses (using sed and awk on a copy of the CSV).
- Develop validation and standardization checks before the datasets are merged.

Student A - Scan-Data Construction and Defect Injection. Student A must individually:

- Construct the Hosts and Ports/Services CSV tables and add controlled test defects.
- Explain the intended schema and the defects introduced for testing.

Student B - Scan-Data Validation and Standardization. Student B must individually:

- Construct the Services, OS/Versions and Enumeration CSV tables (nmap -O, enum4linux -a and nbtscan for Windows, adb shell getprop for Android, rmi-dumpregistry for Java, ncp-serverinfo for NetWare, sdptool browse for Symbian-type Bluetooth devices).
- Implement validation and standardization checks for required columns, valid IP and port ranges and consistent service names using awk, grep -E, grepcidr and a service-name lookup file.

Joint Design Question - Together answer:

- If you were designing a scan-data preparation and validation pipeline for a large authorized assessment, which validation checks would you apply first, and why?
- Both students must be able to defend the answer.

Answer

Shared Group Work

2.1 Scans run in the lab (commands) and how they become CSV

```
nmap -sn 192.168.56.0/24 -oG raw/sweep.gnmap                    # ping sweep    (alt: fping -a -g 192.168.56.0/24)
nmap -sS -sV 192.168.56.0/24 -oG raw/tcp_sv.gnmap -oX raw/tcp_sv.xml    # SYN scan + versions nmap -sU --
top-ports 20 192.168.56.0/24 -oG raw/udp.gnmap              # UDP
nmap -sT -p 21,22,23,25,80,139,443,445,3306,5555 -oG raw/p10.gnmap 192.168.56.0/24
nmap -O 192.168.56.0/24 -oG raw/os.gnmap
```

The grepable files are converted to CSV by sl_import_gnmap (awk). Result of the import step on the
lab-format TCP file:

```
$ sl_import_gnmap raw/tcp_sv.gnmap work/q2 S1 && wc -l work/q2/*.csv  1
work/q2/hosts.csv
 23 work/q2/ports.csv
 23 work/q2/services.csv
 47 total

$ head -3 raw/tcp_sv.gnmap | cut -c1-150
# Nmap 7.94 scan initiated as: nmap -sS -sV -oG tcp_sv.gnmap 192.168.56.0/24
Host: 192.168.56.20 (dvwa-web)  Ports: 22/open/tcp//ssh//OpenSSH 6.6.1p1 Ubuntu/, 80/open/tcp//http//Apache httpd 2...
Host: 192.168.56.21 (msf-lab)  Ports: 21/open/tcp//ftp//vsftpd 2.3.4/, 22/open/tcp//ssh//OpenSSH 4.7p1 Debian/, 23/...

hosts           clean=10    with  defects=12
ports           clean=28    with  defects=39
services      clean=28      with  defects=31
os              clean=10    with  defects=10
enum          clean=20   with defects=21

$ diff <(cat work/clean_src/ports.csv) <(cat work/defect/ports.csv) | grep '^>' >
192.168.56.20,80,tcp,,S1
> 192.168.56.30,445,TCP,open,S1
> 192.168.56.21,22,tcp,open,S1
> 192.168.56.21,22,tcp,open,S1
... (25 more lines in evidence/q2_construct.txt)
```

Student A - Scan-Data Construction and Defect Injection

2.2 Intended schema (see Q1) and the injected defects

Student A built hosts.csv, ports.csv and services.csv with sl_import_gnmap, then copied them and injected the following defects with sed and awk (script inject_defects.sh). The original files were never changed.

D1	Missing value (blank state)	ports	sed s/.../,,S1/ on 192.168.56.20,80	Review: MISSING_STATE
D2	Duplicate host-port records (x2)	ports	awk copy of 192.168.56.21,22 appended twice	Duplicate: first kept, 2 removed
D3	Upper-case protocol 'TCP'	ports	sed on 192.168.56.30,445	Auto-corrected to tcp
D4	Out-of-scope addresses 10.0.0.5,	ports, hosts, services	cat >> append	Reject: OUT_OF_SCOPE
D5	Invalid address 192.168.56.256	ports	append	Reject: BAD_IP
D6	Invalid ports 0, 65536, 'abc'	ports	append	Reject: BAD_PORT
D7	Network/broadcast address	ports	append	Review (valid format, but

	192.168.56.255			not a normal host)
D8	Boundary-valid ports 1 and 65535	ports	append	Accept (must NOT be rejected)
D9	Inconsistent service names www, HTTP, 'ssh ', MySQL	services	sed	Standardised to http, ssh, mysql
D10	Blank service name (9100) and unknown name ms-wbt-server	services	sed / printf	Review: UNKNOWN_SERVICE_NAME
D11	Service row with no port row (192.168.56.30,8443)	services	printf append	Review: SERVICE_WITHOUT_PORT
D12	Blank hostname and duplicate host row	hosts	sed / append	Hostname -> 'unknown'; duplicate removed
D13	Enumeration row for unknown host 192.168.56.99	enum	append	Review: ORPHAN_NO_HOST

Clean versus defective row counts, and the exact differences:

```
> 10.0.0.5,80,tcp,open,S1
> 172.16.4.9,22,tcp,open,S1
> 192.168.56.256,80,tcp,open,S1
> 192.168.56.20,0,tcp,open,S1
> 192.168.56.20,65536,tcp,open,S1
> 192.168.56.20,abc,tcp,open,S1
> 192.168.56.255,80,tcp,open,S1
> 192.168.56.20,65535,tcp,open,S1
> 192.168.56.21,1,tcp,open,S1


$ diff work/clean_src/services.csv work/defect/services.csv | grep '^[<>]' <
192.168.56.20,80,tcp,http,Apache httpd,2.4.7
< 192.168.56.20,3306,tcp,mysql,MySQL,5.5.62
> 192.168.56.20,80,tcp,www,Apache httpd,2.4.7
> 192.168.56.20,3306,tcp,MySQL,MySQL,5.5.62
< 192.168.56.21,22,tcp,ssh,OpenSSH,4.7p1 Debian
> 192.168.56.21,22,tcp,ssh ,OpenSSH,4.7p1 Debian
< 192.168.56.21,80,tcp,http,Apache httpd,2.2.8
> 192.168.56.21,80,tcp,HTTP,Apache httpd,2.2.8
< 192.168.56.80,9100,tcp,jetdirect,,
> 192.168.56.80,9100,tcp,,,
> 192.168.56.31,3389,tcp,ms-wbt-server,,
> 192.168.56.30,8443,tcp,https,,
> 10.0.0.5,80,tcp,www,,
```

Student B - Scan-Data Validation and Standardization

2.3 Services, OS/Versions and Enumeration tables

Table	Tool and command	Extraction rule (awk)
services.csv	nmap -sV	Split each Ports entry on '/': port/state/proto//service//version/; product = words before the first version number
os.csv	nmap -O	ip, family, guess, accuracy
enum - Windows	nbtscan -r 192.168.56.0/24 ; enum4linux -a 192.168.56.30	NetBIOS name, 'Domain Name:', user: [..], Sharename lines
enum - Android	adb connect 192.168.56.40:5555 ; adb shell getprop	[ro.build.version.release]: [9] -> key=value
enum - Java	nmap -p 1099 --script rmi-dumpregistry 192.168.56.50	bound names under rmi-dumpregistry
enum - NetWare	nmap -p 524 --script ncp-serverinfo 192.168.56.60	server name, version, tree

enum - Symbian-type	sdptool browse 00:11:22:33:44:55	Service Name lines

Enumeration table produced by sl_import_enum (first lines): ip,source,key,value, e.g. 192.168.56.40,adb_getprop,ro.build.version.release,9 ; 192.168.56.60,ncp-serverinfo,server_version,NetWare 6.5 ; 192.168.56.70,sdptool,service_name,OBEX Object Push.

2.4 Validation and standardization checks (with results on the defective data)

```
$ head -n1 work/defect/ports.csv | grep -Fx 'ip,port,proto,state,scan_id' && echo HEADER_OK
ip,port,proto,state,scan_id
HEADER_OK

$ awk -F, 'NR>1' work/defect/ports.csv | cut -d, -f1 | grep -Evc '^((25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])\.){...
1

$ awk -F, 'NR>1' work/defect/ports.csv | cut -d, -f1 | grep -Ev '^((25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])\.){3...
192.168.56.256

$ awk -F, 'NR>1' work/defect/ports.csv | cut -d, -f2 | grep -Ev '^([1-9][0-9]{0,3}|[1-5][0-9]{4}|6[0-4][0-9]{3}|65[...
0
65536
abc

$ awk -F, 'NR>1 && $4==""' work/defect/ports.csv          # missing required value
192.168.56.20,80,tcp,,S1

$ # scope check (grepcidr equivalent):   grepcidr 192.168.56.0/24 file    -> awk version used here

$ awk -F, 'function n(x,a){split(x,a,".");return ((a[1]*256+a[2])*256+a[3])*256+a[4]}
BEGIN{lo=n("192.168.56.0")} N...
OUT_OF_SCOPE  10.0.0.5
OUT_OF_SCOPE  172.16.4.9
OUT_OF_SCOPE  192.168.56.256

$ cut -d, -f4 work/defect/services.csv | sed 1d | tr 'A-Z' 'a-z' | sed 's/ *$//' | sort -u | grep -vxFf <(cut -d, -...

ms-wbt-server
```

The same checks are packaged in scanlib.sh (sl_check_header, sl_validate_hosts, sl_validate_keys,
sl_std_services). grepcidr was not installed in the sandbox; its job (is the IP inside 192.168.56.0/24?)
is done by an awk integer comparison, and with grepcidr installed the command is grepcidr 192.168.56.0/24 file.

Joint Design Question - which validation checks first, and why?

1. Schema and header check (cheapest, fatal): if columns are missing or the delimiter is wrong, nothing after it can be trusted. The pipeline stops with exit code 2 (see Q6).

2. Scope check (IP inside the authorized CIDR): this is a legal and ethical control, not just data quality. Out-of-scope rows (10.0.0.5, 172.16.4.9) must be separated at the very start so they are never merged, reported or re-scanned. It is also one of the cheapest checks.

3. IP and port validity: 192.168.56.256, port 0 and 65536 are impossible values; rejecting them early
stops bad keys reaching the joins.

4. Required values / types: blank state or missing key cannot be guessed, so they go to review.

5. Duplicates: after keys are known to be valid, remove repeated ip+port+proto rows.

6. Service-name standardisation last: it needs valid, de-duplicated rows and is the most expensive (lookup).

Principle: check the cheapest and most severe things first, and make the first checks the ones that protect scope and prevent wasted effort. In a large assessment (thousands of hosts) this order also
reduces work because most rejected rows are removed in the first two cheap passes.

Q3 - Build a Scan-Data Editing and Cleaning Module (7 marks)

Question

Shared Group Work - Together, the pair must:

- Together, develop reusable shell operations (Bash functions using sed, awk, sort -u) for missing values, inconsistent service names and duplicate host-port records.
- Show before-and-after data (diff, wc -l, head) and test valid, invalid and boundary cases (ports 0, 1, 65535, 65536; addresses 192.168.56.255 and 192.168.56.256).
- Distinguish records that can be corrected automatically from records that require manual review.

Student A - Scan-Data Cleaning and Correction Rules. Student A must individually:

- Lead cleaning operations for missing values, duplicate host-port records and malformed addresses.
- Explain the rules used for each automatic correction.

Student B - Service-Name Standardization and Test Validation. Student B must individually:

- Lead the rules for inconsistent service names (lookup file mapping variants such as www, HTTP, http-alt to one standard name) and manual-review flags; design verification tests.
- Verify that cleaning does not silently remove meaningful scan findings (record counts; every removed row must appear in a rejected or review file).

Joint Conclusion - Together answer:

- Why is automated scan-data cleaning alone insufficient to guarantee the accuracy of an assessment's findings?
- Both students must be able to justify their answers with suitable examples from their data-cleaning and validation tasks.

Answer

Shared Group Work

3.1 Reusable shell operations (scanlib.sh)

sl_trim	sed	Removes CRLF and extra spaces around commas	0
sl_lower_col IN OUT COL	awk	Lower-cases protocol/state columns	0 / 1 missing file
sl_fill_missing IN OUT COL DEFAULT	awk	Fills a blank descriptive column (e.g. hostname) with 'unknown'	0 / 1
sl_dedupe IN OUT REJ KEYS	awk, sort -u	Keeps the first row per key, writes removed rows to REJ	0 / 1
sl_validate_keys IN OK REJ REV [CIDR]	awk	Splits rows into accepted / rejected / review; checks input = ok + rej + rev	0 / 1 / 3 count mismatch
sl_std_services IN LOOKUP OUT REV	awk	Maps service-name variants through the lookup; unknown names go to review	0 / 1

3.2 Before-and-after evidence

```
$ sl_validate_keys work/defect/ports.csv work/q3/p_ok.csv work/q3/p_rej.csv work/q3/p_rev.csv validate_keys:
in=39 ok=30 rejected=6 review=3


$ wc -l work/defect/ports.csv work/q3/p_ok.csv work/q3/p_rej.csv work/q3/p_rev.csv 40
work/defect/ports.csv
 31 work/q3/p_ok.csv
  7 work/q3/p_rej.csv
  4 work/q3/p_rev.csv
 82 total


$ head -4 work/q3/p_rej.csv; head -3 work/q3/p_rev.csv
reason,line,row
OUT_OF_SCOPE,32,10.0.0.5,80,tcp,open,S1
OUT_OF_SCOPE,33,172.16.4.9,22,tcp,open,S1
BAD_IP,34,192.168.56.256,80,tcp,open,S1
reason,line,row
MISSING_STATE,3,192.168.56.20,80,tcp,,S1
BAD_PROTO,15,192.168.56.30,445,TCP,open,S1


$ diff <(sort work/q3/p_ok.csv) <(sort work/q3/p_dd.csv) 10,11d9
< 192.168.56.21,22,tcp,open,S1
< 192.168.56.21,22,tcp,open,S1


$ sort -u work/q3/p_ok.csv | wc -l; wc -l < work/q3/p_dd.csv 29
29


$ cat work/q3/p_dups.csv
reason,line,row
DUPLICATE,28,192.168.56.21,22,tcp,open,S1
DUPLICATE,29,192.168.56.21,22,tcp,open,S1


$ diff <(cut -d, -f1-4 work/defect/services.csv) <(cut -d, -f1-4 work/q3/s_std.csv) | head -14 3,4c3,4
< 192.168.56.20,80,tcp,www
< 192.168.56.20,3306,tcp,MySQL
---
> 192.168.56.20,80,tcp,http
> 192.168.56.20,3306,tcp,mysql
6c6
< 192.168.56.21,22,tcp,ssh
---
> 192.168.56.21,22,tcp,ssh
9c9
< 192.168.56.21,80,tcp,HTTP
---
> 192.168.56.21,80,tcp,http


$ cat work/q3/s_rev.csv
reason,line,row
UNKNOWN_SERVICE_NAME,23,192.168.56.80,9100,tcp,,,
UNKNOWN_SERVICE_NAME,30,192.168.56.31,3389,tcp,ms-wbt-server,,
```

sort -u and sl_dedupe agree (29 and 29 rows), but sl_dedupe also records the 2 removed duplicates
in a file, which sort -u alone does not.

3.3 Boundary, valid and invalid cases

```
input row                                verdict
192.168.56.20,0,tcp                      REJECT (BAD_PORT)
192.168.56.20,1,tcp                     ACCEPT
192.168.56.20,65535,tcp                 ACCEPT
192.168.56.20,65536,tcp                  REJECT (BAD_PORT)
192.168.56.255,80,tcp                    REVIEW
(NETWORK_OR_BROADCAST_ADDR) 192.168.56.256,80,tcp                    REJECT
(BAD_IP)
192.168.56.0,80,tcp                      REVIEW
(NETWORK_OR_BROADCAST_ADDR) 192.168.56.20,80,udp
ACCEPT
,80,tcp                                 REJECT (MISSING_KEY)


reconcile: input=9 accepted=3 rejected=4 review=2
```

Note: 192.168.56.255 and 192.168.56.0 are syntactically valid but are the broadcast and network addresses of the /24 - a person must decide whether they are a real device, so they go to review, not
to reject.

3.4 Automatic correction versus manual review

Situation	Decision	Why
Extra spaces, CRLF, 'TCP' instead of tcp	Automatic	Meaning cannot change
Service name variants www / HTTP / MySQL / 'ssh '	Automatic (lookup)	Many-to-one mapping known in the lookup file
Blank hostname	Automatic ('unknown')	Descriptive field, not a key
Exact duplicate ip+port+proto	Automatic (keep first, log removed)	Same fact twice
Blank state (open or closed?)	Manual review	A wrong guess changes the finding
192.168.56.255 / .0	Manual review	Valid format, unusual host
Unknown service names (ms-wbt-server, blank)	Manual review	Lookup cannot decide
192.168.56.256, port 0 / 65536 / 'abc', out-of-scope IP	Rejected (kept in reject file)	Cannot be correct or must not be used

Student A - Cleaning and Correction Rules

Student A led missing values, duplicates and malformed addresses. Rules and reasons:

Rule	Implementation	Reason / safety
A1 Missing descriptive value -> 'unknown'	sl_fill_missing (awk) on hostname/service only	Never applied to key columns or state
A2 Missing key (ip, port, proto) -> reject	sl_validate_keys: MISSING_KEY	A row without a key cannot be joined
A3 Duplicate host-port -> keep first	sl_dedupe keys 1,2,3; removed rows -> rejected file as DUPLICATE	Identical facts; evidence kept
A4 Malformed address -> reject, never 'fixed'	sl_validate_keys: BAD_IP (octet > 255, wrong number of parts)	Guessing 192.168.56.256 -> .25 would invent a host
A5 Network/broadcast address -> review	NETWORK_OR_BROADCAST_ADDR	Valid format but needs a human

Student B - Service-Name Standardization and Test Validation

Lookup file lookup/service_names.csv (variant,standard) - excerpt:

```
www,http
http,http
http-alt,http
www-http,http
microsoft-ds,smb
cifs,smb
openssh,ssh
telnetd,telnet
domain,dns
```

Rules: names are lower-cased and trimmed before lookup; the original is kept in the column service_raw (traceability); a name that is not in the lookup is written as REVIEW and listed in the review file. Design of verification tests:

Test	Method	Result on the lab run
T1 Record counts	input = ok + rejected + review	39 = 30 + 6 + 3 (ports), OK; function

		returns 3 if not equal
T2 No silent removal	Every removed row has a reason line in a reject/review file	Ports: 6 rejected + 2 duplicates + 3 review rows all listed
T3 Boundary ports	1 and 65535 accepted; 0 and 65536 rejected	As expected (section 3.3)
T4 Standardisation is reversible	service_raw column keeps the original	www -> http, original 'www' still visible
T5 Idempotence	Running cleaning on clean data changes nothing	Clean raw run: 0 rejected, 0 review, 28 = 28

Joint Conclusion - why automated cleaning alone is not enough

- **It cannot know the truth, only the format.** The row 192.168.56.20,80,tcp,(blank) is well-formed but the state is missing; open or closed changes the finding, so the cleaner sent it to review rather than guess.
- **Valid does not mean real.** 192.168.56.255 passes the dotted-quad rule but is the broadcast address; only a person can decide whether it is a device.
- **Standardisation can hide information.** Mapping http-alt, www and HTTP to one name makes counting easy, but it can hide that 8080/http-alt is a non-standard web port on a Symbian emulator. That is why service_raw is kept.
- **Unknown cases remain.** ms-wbt-server and the blank service on 9100 were not in the lookup, so a human decides.
- **De-duplication assumes the duplicate is the same fact.** If the two rows came from different scans with different states, keeping the first could discard a change.
- **Tools can be wrong.** nmap -sV banners such as 'Samba smbd 3.X' are guesses; cleaning cannot check them against reality. Findings must be verified in the lab.

Q4 - Create a Scan-Result Reshaping Pipeline Using Melting and Casting (7 marks)

Question

Shared Group Work - Together, the pair must:

- Together, create a wide-format scan dataset (for example hosts as rows and ports as columns, built from nmap -p 21,22,23,25,80,139,443,445,3306,5555 -oG output) and reshape it into long format (host, port, state) using awk.
- Create a cast operation (long to wide) for a host-by-service or subnet-by-service summary using awk arrays, datamash crosstab or sort | uniq -c.
- Identify the identifiers, dimensions and measured variables and validate that the reshape preserves the information (row counts and checksum of the round-trip output).

Student A - Scan-Result Reshaping and Transformation. Student A must individually:

- Implement the wide-to-long transformation and identify the identifier and measured columns.
- Explain why the selected long format suits the required analysis.

Student B - Summary Casting and Result Validation. Student B must individually:

- Create the cast/summary output (such as open-ports-per-host) and compare its totals with the source data.
- Explain how the validation demonstrates that the transformation preserved the scan information.

Joint Conclusion - Together answer:

- Why does a more detailed scan result not always produce a clearer or more useful assessment summary?

- Both students must be able to justify their decision using theoretical complexity (a full -p- scan probes 65,535 ports per host per protocol; a long table grows as hosts x ports) and experimental results (time nmap, wc -l).

Answer

Shared Group Work

4.1 Wide format built from the -p 21,...,5555 grepable scan

```
nmap -p 21,22,23,25,80,139,443,445,3306,5555 192.168.56.0/24 -oG raw/p10.gnmap
sl_wide_from_gnmap raw/p10.gnmap wide.csv 21,22,23,25,80,139,443,445,3306,5555 hosts.csv
```

Ports not listed by nmap (closed, ignored) are written as 'closed'; hosts with no listed port are added
from the hosts table so no live host is lost.

Role	Columns
Identifier	ip (one per host)
Dimension (variable name)	port (p21 ... p5555)
Measured variable	state (open / closed / filtered)

```
$ cat work/q4/wide.csv | column -s, -t 2>/dev/null || cat work/q4/wide.csv
ip,p21,p22,p23,p25,p80,p139,p443,p445,p3306,p5555
192.168.56.1,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.20,closed,open,closed,closed,open,closed,closed,closed,open,closed
192.168.56.21,open,open,open,open,open,open,closed,open,open,closed
192.168.56.30,closed,closed,closed,closed,closed,open,open,closed,open,closed,closed
192.168.56.31,closed,closed,closed,closed,closed,closed,open,closed,open,closed,closed
192.168.56.40,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed,open
192.168.56.50,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.60,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.70,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed,closed
192.168.56.80,closed,closed,closed,closed,open,closed,closed,closed,closed,closed


$ head -6 work/q4/long.csv; wc -l work/q4/wide.csv work/q4/long.csv ip,port,state
192.168.56.1,21,closed
192.168.56.1,22,closed
192.168.56.1,23,closed
192.168.56.1,25,closed
192.168.56.1,80,closed
 11 work/q4/wide.csv
 101 work/q4/long.csv
 112 total


$ awk -F, 'NR>1 && $3=="open"' work/q4/long.csv | wc -l
18


$ sha256sum work/q4/wide.csv work/q4/wide2.csv | cut -c1-20,65-
631d75b45d4c3bc88c09  work/q4/wide.csv
631d75b45d4c3bc88c09  work/q4/wide2.csv


$ cmp work/q4/wide.csv work/q4/wide2.csv && echo ROUND_TRIP_IDENTICAL
ROUND_TRIP_IDENTICAL
```

Student A - Wide-to-Long Transformation

```
# sl_melt : wide -> long
awk -F, 'NR==1{for(i=2;i<=NF;i++){p[i]=$i; sub(/^p/,"",p[i])}; print "ip,port,state"; next}     {for(i=2;i<=NF;i++)
print $1","p[i]","$i}' wide.csv > long.csv
```

Result: 10 hosts x 10 ports = 100 long rows (101 lines with header; wide has 11 lines). The identifier
is ip, the port number is moved from the column name into a value, and state is the measured column.

Why the long format suits the analysis: (1) one row = one observation, so filtering (state==open), counting and sorting are one-line awk/sort commands; (2) it joins directly with services.csv on ip+port; (3) a new port or a UDP scan adds rows, not columns; (4) several sessions can be appended
and compared; (5) the wide format is only good for a human reading a small grid.

Student B - Summary Casting and Result Validation

```
$ cat work/q4/open_per_host.csv
ip,open_ports
192.168.56.1,0
192.168.56.20,3
192.168.56.21,8
192.168.56.30,3
192.168.56.31,2
192.168.56.40,1
192.168.56.50,0
192.168.56.60,0
192.168.56.70,0
192.168.56.80,1

 =", s}'$ awk -F, 'NR>1{s+=$2} END{print "sum of open ports per host =", s}' work/q4/open_per_host.csv
sum of open ports per host = 18

$ awk -F, 'NR>1 && $3=="open"' work/q4/long.csv | wc -l 18

$ awk -F, 'NR>1 && $3=="open"{print $2}' work/q4/long.csv | sort -n | uniq -c | sort -rn 4 80
      3 445
      3 139
      2 3306
      2 22
      1 5555
      1 25
      1 23
      1 21

$ awk -F, 'NR>1 && $3=="open"{split($1,a,"."); c[a[1]"."a[2]"."a[3]".0/24","$2]++} END{for(k in c)print k, c[k]}' work/q4...
192.168.56.0/24,139 3
192.168.56.0/24,21 1
192.168.56.0/24,22 2
192.168.56.0/24,23 1
192.168.56.0/24,25 1

$ head -3 work/q4/host_by_service.csv | cut -c1-120
192.168.56.1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0
192.168.56.20,0,0,0,0,0,0,1,0,0,0,0,0,0,0,1,0,0,1,0
192.168.56.21,1,1,0,0,0,1,0,1,0,1,0,1,1,1,0,1,1
```

Validation: (1) open rows in the long table = 18; (2) sum of open_ports per host = 18; the totals agree. (3) Round trip wide -> long -> wide gives the same sha256 checksum (631d75b4...) and cmp
reports no difference, and the row counts are 11 -> 101 -> 11. If a row had been dropped or a state
changed in either direction, one of these three checks would have failed. Together they show the transformation preserved the scan information.

Joint Conclusion - more detail is not always clearer

Theory. A full scan (-p-) probes 65,535 ports per host per protocol, so probes = hosts x 65,535 (x2 if
UDP is included). The long table grows as hosts x ports, but almost all rows say 'closed'. Experiment
(reshape of synthetic wide tables, same awk melt):

Hosts	Ports probed	Long rows (hosts x ports)	File size	Melt time	Open rows
10	10	100	4KB	0.00s	10
250	10	2,500	48KB	0.00s	250
20	1000	20,000	400KB	0.01s	30

10	65535	655,350	14036KB	0.23s	239

For 10 hosts, a full-range scan gives 655,350 long rows (about 14 MB) to find about 270 open ports (0.04%). Extending to 250 hosts is 16,383,750 rows. At an assumed 1,000 probes per second (nmap --min-rate 1000) that is about 16,400 s (4.5 hours) per protocol; at 100 probes per second about 45 hours (calculated, not measured; in the lab record real values with time nmap -p- --min-rate 1000 192.168.56.20 and wc -l).

Conclusion: the open-ports-per-host summary has one row per host (10 rows) whatever the scan size, and that is what a manager can read. More detail raises cost, storage and noise, and can bury the few important rows; so detail must match the question (targeted ports first, full range only where needed).

Q5 - Design a Multi-Stage Scan-Data Merging Strategy (6 marks)

Question

Shared Group Work - Together, the pair must:

- Together, combine the Hosts, Ports, Services, OS/Versions and Enumeration datasets into a single master scan table using join, awk arrays and sort.
- Identify the keys (such as IP address and port) , select suitable merge types (join inner, -a1 left, -a1 -a2 full outer, -v unmatched) and test at least two alternatives.
- Detect unmatched records (join -v, comm -23), duplicated matches (sort | uniq -d) and unexpected row multiplication after merging (wc -l before and after).

Student A - Host-Service Integration and Join Validation. Student A must individually:

- Lead the Services-to-Hosts and Ports-to-Hosts joins; document the key choices.
- Check the record counts and ensure the joins do not create unintended row multiplication.

Student B - Version-Enumeration Integration and Exception Analysis. Student B must individually:

- Lead the OS/Versions and Enumeration integration (Windows SMB/NetBIOS, Android ADB, Java RMI, NetWare NCP findings); compare alternative join types.
- Help verify the final master scan table and document exceptions.

Cross-Understanding Requirement:

- Student B must be able to explain: Why can different join types produce different record counts, and how can unmatched hosts or services be detected?
- Student A must be able to explain: What experimental evidence demonstrates that the Services-to-Hosts and Ports-to-Hosts joins are correct?
- This deliberately tests understanding across the assigned responsibilities.

Answer

Shared Group Work

5.1 Merge plan, keys and merge types

1	Ports	Services	ip:port:proto	Left (-a1)	Keep every open port even if no service was identified
2	Hosts	Result of 1	ip	Left (-a1), hosts on the left	A live host with no open port must still appear in the master
3	Result of 2	OS/Versions	ip	Left	Host-level attribute, 0 or 1 row per host
4	Result of 3	Enumeration	ip	Left	Enumeration has many
		(aggregated)			rows per host; collapse first to avoid multiplication

Master columns: ip, hostname, status, port, proto, state, service, product, version, os_family, os_guess, enum_summary. Implemented in sl_merge (join with composite key column, sort, awk array for enumeration aggregation). join requires both inputs sorted on the key with the same locale
(LC_ALL=C is set).

Student A - Services-to-Hosts and Ports-to-Hosts joins

Key choice: ports and services are keyed by ip:port:proto (not ip:port) because the same number can
occur on tcp and udp; hosts are keyed by ip. Test set: ports 28 rows; services made faulty (two services removed, one orphan service added).

```
$ wc -l < work/q5/p.k; wc -l < work/q5/s.k              # ports rows ; services rows (2 services missing, 1 orphan added)
28
27

$ join -t, work/q5/p.k work/q5/s.k | wc -l                  # inner 26

$ join -t, -a1 work/q5/p.k work/q5/s.k | wc -l              # left (ports kept) 28

$ join -t, -a1 -a2 work/q5/p.k work/q5/s.k | wc -l          # full outer 29

$ join -t, -v1 work/q5/p.k work/q5/s.k | cut -d, -f1        # ports without a service row 192.168.56.20:80:tcp
192.168.56.31:445:tcp

$ join -t, -v2 work/q5/p.k work/q5/s.k | cut -d, -f1        # service rows without a port row
192.168.56.30:8443:tcp

$ comm -23 <(cut -d, -f1 work/q5/p.k) <(cut -d, -f1 work/q5/s.k)
192.168.56.20:80:tcp
192.168.56.31:445:tcp

$ comm -13 <(cut -d, -f1 work/q5/p.k) <(cut -d, -f1 work/q5/s.k)
192.168.56.30:8443:tcp

$ cut -d, -f1 work/q5/s_dup.k | sort | uniq -d          # duplicated join key in services 192.168.56.21:22:tcp

$ join -t, -a1 work/q5/p.k work/q5/s_dup.k | wc -l          # left join with a duplicated key: row multiplication (28 ->
29)
29
```

The counts show how the merge type changes the result: inner 26, left 28, full outer 29. Unmatched
records were found with join -v1 / -v2 and confirmed with comm -23 / -13. A duplicated key (192.168.56.21:22:tcp in services) was detected with sort | uniq -d, and a left join with that duplicate gave 29 instead of 28 rows - unintended row multiplication.

Ports-to-Hosts join

```
$ wc -l < work/q5/h.k; wc -l < work/q5/pt.k              # hosts ; port rows (hosts .50 and .60 have no port rows)
10
26

$ join -t, work/q5/h.k work/q5/pt.k | wc -l                  # inner: unscanned/closed hosts silently vanish 26

$ join -t, -a1 work/q5/h.k work/q5/pt.k | wc -l              # left (hosts kept): one row per port + one row per host witho...
28
```

```
$ join -t, -v1 work/q5/h.k work/q5/pt.k                    # hosts with no port rows (detected) 192.168.56.50
192.168.56.60

$ join -t, -v2 work/q5/h.k work/q5/pt.k | wc -l             # port rows whose host is unknown 0

$ join -t, -a1 <(cut -d, -f1,2 work/q5/pt.k | sort -t, -k1,1) work/q5/enum_raw.k | wc -l    # naive join with un-aggregated...
58

$ cut -d, -f1 work/q5/enum_raw.k | uniq -c | head -4
      6 192.168.56.30
      4 192.168.56.31
      3 192.168.56.40
      2 192.168.56.50

$ join -t, -a1 <(cut -d, -f1,2 work/q5/pt.k | sort -t, -k1,1) work/q5/enum_agg.k | wc -l    # enumeration collapsed to 1 ro...
26
```

Student B - Version-Enumeration integration and exception analysis

OS/Versions is one row per host, so it joins 1:1 and cannot multiply rows. Enumeration findings are many rows per host (6 for 192.168.56.30). A naive join of ports with raw enumeration rows gave **58 rows**; collapsing enumeration to one row per host (source:key=value;...) first gave **26 rows**, the same
number as the ports. Alternative join types for enumeration compared on the master:

Left (chosen)	28	All ports kept; enum_summary = NA for hosts not enumerated
Inner	11	17 port rows of hosts without enumeration (e.g. 192.168.56.21, which has 10 findings of interest) disappear
Naive left, raw enum rows	about 60	Row multiplication; counts of ports are wrong

Enumeration coverage: Windows (SMB/NetBIOS) 192.168.56.30, .31; Android ADB .40; Java RMI .50;
NetWare NCP .60; Symbian-type sdptool .70. Master verification and exceptions:

```
$ wc -l < work/clean_src/ports.csv; wc -l < work/out_clean/master.csv          # (header included) ports rows vs master rows
29
29

$ cut -d, -f1,4,5 work/out_clean/master.csv | sed 1d | sort | uniq -d | wc -l    # duplicated ip,port,proto keys in master
0

$ awk -F, 'NR==FNR{if(FNR>1)s[$1":"$2":"$3]=$4;next} FNR>1&&($1":"$4":"$5) in s&&s[$1":"$4":"$5]!=$7{bad++} END{print
"ser...
service mismatches: 0

$ awk -F, 'NR==FNR{if(FNR>1)o[$1]=$2;next} FNR>1&&o[$1]!=$10{bad++} END{print "os mismatches:", bad+0}'
work/out_clean/cle...
os mismatches: 0

$ cut -d, -f1 work/out_clean/master.csv | sed 1d | sort -u | wc -l; sed 1d work/out_clean/clean/hosts.csv | wc -l    # h...
10
10

$ awk -F, 'NR>1&&$7=="NA"' work/out_clean/master.csv | wc -l    # master rows with no service match 0

$ awk -F, 'NR>1 && $12!="NA"{print $1}' work/out_clean/master.csv | sort -u | tr '\n' ' ' 192.168.56.30
192.168.56.31 192.168.56.40 192.168.56.50 192.168.56.60 192.168.56.70
```

```
$ awk -F, 'NR>1 && $12!="NA"' work/out_clean/master.csv | wc -l        # rows an INNER join with enumeration would keep
(28 ...
11



-- exceptions (defective run) --
$ wc -l work/out_def/review/*.csv | sed 's#work/out_def/review/##' 2
enum_orphans.csv
 1 hosts.csv
 3 ports.csv
 1 serv_orphans_host.csv
 4 serv_without_port.csv
 1 services.csv
 3 services_names.csv
 15 total


$ cut -d, -f1-3 work/out_def/review/serv_without_port.csv | head -5
reason,line,row
SERVICE_WITHOUT_PORT,3,192.168.56.20
SERVICE_WITHOUT_PORT,30,192.168.56.31
SERVICE_WITHOUT_PORT,31,192.168.56.30
```

Exceptions documented: (1) .20:80 had a missing state - removed from the master and listed in review/ports.csv, so its service row became 'service without port'; (2) 192.168.56.30:8443 and .31:3389 had service rows without port rows; (3) 192.168.56.99 had an enumeration row but is not a
known host; (4) .80:9100 has the service name REVIEW.

Cross-Understanding Answers

Student B - why join types give different counts and how to detect unmatched rows. Inner keeps only keys found on both sides (26); left keeps all left rows and fills the missing right side (28); full outer also keeps right-only rows (29); -v prints only the unmatched rows from one side. The counts differ exactly by the unmatched rows. Detection: join -v1 (left-only), join -v2 (right-only), comm -23 / -13 on the sorted key lists, and a wc -l comparison before and after the join.

Student A - evidence that the joins are correct. (1) Row count: master has 28 rows = ports rows, so no row was lost or multiplied; (2) sort | uniq -d on ip,port,proto in the master gives 0 duplicate keys; (3) a key-by-key awk comparison shows 0 service mismatches and 0 OS mismatches against the source tables; (4) the 10 hosts in the master equal the 10 clean hosts; (5) the deliberate faults (missing service, orphan, duplicate key) were detected by join -v and uniq -d.

Q6 - Create an Automated Scan-File Reading and Writing Pipeline (7 marks)

Question

Shared Group Work - Together, the pair must:

- Together, read source scan-output files (CSV or Nmap results exported to CSV from XML or grepable output using xsltproc / awk), check the required columns and import them into CSV tables.
- Create a naming and versioning strategy that avoids accidentally overwriting earlier scan results (for example subnet_scantype_YYYYMMDD-HHMMSS_vN.csv, set -o noclobber, sha256sum checksum file).

Student A - Scan-File Import and Schema Validation. Student A must individually:

- Implement scan-file import and schema/type checks before processing (header match, numeric port column, valid IP pattern).
- Demonstrate how the pipeline identifies missing required columns or unexpected formats.

Student B - Result Export, Versioning and Read-Back Validation. Student B must individually:

- Implement export of the cleaned, reshaped and master scan datasets; define the output naming and versioning.

- Compare the re-imported data with the written outputs (diff, cmp, sha256sum -c, wc -l).

Answer

Shared Group Work

6.1 Reading scan files

Input	Method	Notes
nmap -oG grepable	awk (sl_import_gnmap): split on tab, then on ', ' and '/'	Used in this work; one line per host
nmap -oX XML	xsltproc with nmap.xsl gives HTML; for CSV extract fields with xmlstarlet or convert XML -> grepable with an XSLT/awk step	xsltproc was not installed in the sandbox; command: xsltproc nmap.xsl scan.xml -o scan.html
Ready CSV	sl_check_header then sl_trim	Header must equal the schema exactly

6.2 Naming and versioning strategy

Element	Rule
File name	<subnet>_<scantype>_<YYYYMMDD-HHMMSS>_v<N>.csv, e.g. 192.168.56.0-24_master_20261101-101500_v1.csv ('/' becomes '-')
Never overwrite	Redirect is run under set -o noclobber; if the name exists, N increases
Integrity	Every export appends its sha256 to checksums.sha256; sha256sum -c verifies it later
Raw files	Raw scan outputs are kept read-only and never edited; all cleaning works on copies

Student A - Import and Schema Validation

```
$ bash run_pipeline.sh -i work/q6/bad -o work/q6/out1; echo exit_code=$?
scanlib: header mismatch in work/q6/bad/ports.csv: got 'ip,port,proto,state' expected
'ip,port,proto,state,scan_id'
scanlib: FAILED at line 20 (exit 2)
exit_code=2


$ bash run_pipeline.sh -i work/q6/bad -o work/q6/out2; echo exit_code=$?
scanlib: header mismatch in work/q6/bad/ports.csv: got 'ip   port   proto   state   scan_id' expected
'ip,port,proto,state,sca...
scanlib: FAILED at line 20 (exit 2)
exit_code=2


$ awk -F, 'NR>1 && $2 !~ /^[0-9]+$/' work/q6/bad/ports.csv              # numeric port column check
192.168.56.30,one-three-nine,tcp,open,S1


$ bash run_pipeline.sh -i work/q6/bad -o work/q6/out3 | grep -E 'rejected.ports|clean.ports' clean.ports
27
rejected.ports                  1


$ rm -f work/q6/nofile; sl_import_gnmap work/q6/nofile.gnmap work/q6/x S1; echo exit_code=$? scanlib:
missing or unreadable file: work/q6/nofile.gnmap
exit_code=1
```

The pipeline stops with exit code 2 and a clear message for a missing column or the wrong delimiter
(a tab file shows as one long header), exit code 1 for a missing file, and routes a non-numeric port to
the reject file while the other 27 rows continue (clean 27, rejected 1). A valid-IP check (grep -E on the
octets, Q2) is applied in the same stage.

Student B - Export, Versioning and Read-Back Validation

```
$ sl_export work/out_clean/master.csv 192.168.56.0/24 master work/q6/exp
work/q6/exp/192.168.56.0-24_master_20261101-101500_v1.csv
```

```
$ sl_export work/out_clean/master.csv 192.168.56.0/24 master work/q6/exp
```

```
work/q6/exp/192.168.56.0-24_master_20261101-101500_v2.csv


$ (set -o noclobber; echo overwrite > work/q6/exp/192.168.56.0-24_master_20261101-101500_v1.csv) make_evidence2.sh:
line 5: work/q6/exp/192.168.56.0-24_master_20261101-101500_v1.csv: cannot overwrite existing file


$ ls work/q6/exp; cat work/q6/exp/checksums.sha256 | cut -c1-24,65-192.168.56.0-
24_master_20261101-101500_v1.csv
192.168.56.0-24_master_20261101-101500_v2.csv
checksums.sha256
689f4ccfcf6895be213c31d0    192.168.56.0-24_master_20261101-101500_v1.csv
689f4ccfcf6895be213c31d0    192.168.56.0-24_master_20261101-101500_v2.csv


$ cd work/q6/exp && sha256sum -c checksums.sha256
192.168.56.0-24_master_20261101-101500_v1.csv:          OK
192.168.56.0-24_master_20261101-101500_v2.csv: OK


$ sl_import_gnmap /dev/null work/q6/rb S1 2>/dev/null; wc -l < work/q6/exp/192.168.56.0-24_master_20261101-101500_v1.csv; ...
29
29
CMP_IDENTICAL
DIFF_EMPTY


$ cd work/q6/exp && sha256sum -c checksums.sha256 2>&1 | grep -v ': OK'; true
192.168.56.0-24_master_20261101-101500_v1.csv: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

Results: the second export is automatically named _v2 (no overwrite); a direct redirect over an existing file is blocked by noclobber; sha256sum -c reports OK for the untouched files; after a deliberate one-word edit the checksum check reports FAILED; the re-read file matches the master by
cmp, diff and wc -l (29 lines including the header).

Q7 - Develop a Reusable Scanning and Enumeration Function Library (10 marks)

Question

Day-4 problem: solved collaboratively.

- Neither student may work on this question independently and simply combine the answers later. The library is a Bash file (scanlib.sh, loaded with source) whose functions take the subnet, scan type and file names as arguments with default values, return status codes and write errors to stderr.

Stage 1 - Independent Function Design Proposal

- Student A - Validation and Cleaning Function Design: lead the validation and cleaning functions
  for scan data; specify inputs, default arguments and return values. Test the functions independently and document their use.
- Student B - Scan Transformation and File-Handling Function Design: lead the reshaping, merging and file-handling functions; include error handling (set -euo pipefail, trap, missing-file checks). Demonstrate reuse with a different subnet or scan session.
- Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent algorithmic thinking.

Stage 2 - Integrated Pipeline Implementation and Validation

- Student A - Import, Validation and Cleaning Integration: integrate the import, validation and cleaning stages; record the input, rejected and cleaned record counts. Check that invalid inputs are handled consistently.

• Student B - Transformation, Merging and Export Integration: integrate the reshaping, merging and export stages; record the merged and final record counts. Run the workflow again with the same inputs and check reproducibility (identical sha256sum of outputs).
• Each student must document their implementation, test cases, record counts, and validation results. Both students must demonstrate that the integrated pipeline operates correctly and produces consistent results.

Stage 3 - Create and Validate a Reporting Scan Summary

• Student A - Host and Service Reporting and Reconciliation: create per-host and per-service summary reports (awk, sort | uniq -c) and document how they are calculated. Verify the reported totals against the master scan table.
• Student B - Vulnerability-Exposure Reporting and Management Report: create an exposure or service-version risk report (for example outdated services found with nmap -sV, nmap --script vuln or searchsploit on the lab targets) and implement the summary. Explain how the report addresses a management requirement.

Stage 4 - Stress-Test, Refine and Optimize the Scan Pipeline

• Student A - Dataset Scaling and Baseline Benchmarking: generate or prepare scan datasets at the required scales (for example 50 and 5,000 host-port records) and benchmark the baseline (time, /usr/bin/time -v). Record input size, elapsed time and correctness checks.
• Student B - Bottleneck Analysis and Performance Optimization: analyse the measurements, identify a bottleneck and implement one refinement (for example replace a per-line grep loop with a single awk pass, or tune discovery with nmap -T4 --min-rate / masscan). Compare before-and-after performance and explain any limitations.

Stage 5 - Redesign the Pipeline Under a Changed Assessment Requirement

• Student A - Schema Evolution and Architecture Redesign: analyse the impact on the schema, keys, validation and data retention. Propose revised data structures and rules for new scan sources such as UDP services (nmap -sU, key becomes IP + port + protocol) or newly discovered hosts (including Symbian, Java OS, Android and NetWare devices). Retain the impact analysis and explain the revised architecture.
• Student B - Multi-Session Workflow Implementation and Change Validation: implement or modify the multi-session scan/merge workflow and test retained, updated, unmatched and newly discovered hosts (compare sessions with ndiff and comm). Present test evidence, compare with the original pipeline and identify trade-offs.

Stage 6 - Joint Evaluation: Together determine:

• Why the scan-data pipeline requires a structured sequence of import, validation, cleaning, reshaping, merging and export operations.
• How modular functions and systematic transformations improve the correctness, maintainability and efficiency of the scanning workflow.
• What additional costs or challenges arise from validation, error handling, repeated scanning and data integration.
• Under what conditions the proposed pipeline might become inefficient, unreliable or unsuitable for changing assessment requirements.
• Conclude: What characteristics should a security analyst consider when designing a reliable, efficient and maintainable scan-data pipeline for evolving networks?

Answer

Library overview (scanlib.sh and run_pipeline.sh)

scanlib.sh is loaded with source scanlib.sh. Defaults: SL_SUBNET=192.168.56.0/24, SL_STAMP (time stamp, overridable for reproducible file names). Return codes: 0 success, 1 usage or missing file, 2 schema/header error, 3 record-count reconciliation failure (a row was lost), 4 no in-scope data survived (pipeline). All messages go to stderr. run_pipeline.sh uses set -euo pipefail, a trap on ERR

that prints the failing line and exit code, and a trap on EXIT that removes the temporary directory. Usage: run_pipeline.sh [-i INDIR] [-o OUTDIR] [-s SUBNET] [-t SCANTYPE] [-r RAWDIR].

Stage 1 - Independent Function Design Proposals (retained as first written)

Student A - initial proposal (validation and cleaning)

Function (initial)	Inputs / defaults	Return	Initial idea
validate_ip ip	ip	0 valid, 1 invalid	grep -E '^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$'
validate_port port	port	0 / 1	number between 1 and 65535
clean_ports file	file, default ports.csv	0	sort -u the file to remove duplicates; delete bad rows
fix_services file	file	0	sed replace www and HTTP with http

Problems found later (kept honestly): the IP regex accepts 999.1.1.1; 'delete bad rows' loses evidence; sort -u cannot say which row was removed; sed replacements hard-code names instead of using a lookup. Final design: awk octet checks, reject/review files, lookup file, key = ip+port+proto.

Student B - initial proposal (transformation and file handling)

Function (initial)	Inputs / defaults	Return	Initial idea
to_long wide long	two file names	0	awk loop over columns
merge a b	two files	0	join -t, on column 1
save_result file	file	0	cat file > name_v1.csv
set -euo pipefail at the top of scanlib.sh	-	-	stop on any error

Problems found later: join without sorting and without a composite key gives wrong matches; no check for existing files (overwrite); put in set -e in a sourced library would exit the user's shell on any non-zero status, so error handling moved to return codes in the library and set -euo pipefail + trap only in run_pipeline.sh; a left join on raw enumeration multiplied rows (Q5).

Reuse demonstration (Student B): the same pipeline was run for a different subnet/session by changing arguments only (Stage 2).

Stage 2 - Integrated Pipeline Implementation and Validation

Student A - import, validation and cleaning integration (counts)

Run on the defective data (all Q2 defects present). Consistency rules: every invalid input class produces the same kind of outcome - reject file (cannot be right), review file (human decides) or automatic correction - and every row is accounted for.

Table	Input rows	Clean rows	Rejected	Review	Check
hosts	12	10	2 (out of scope, duplicate)	0	12 = 10 + 2
ports	39	29	8 (6 invalid/out of scope + 2 duplicates)	2 (missing state, broadcast)	39 = 29 + 8 + 2
services	31	30	1 (out of scope)	2 name reviews + 3 without port	31 = 30 + 1 (name/without-port rows flagged, not dropped from clean)
os	10	10	0	0	-

enum	21	20	0	1 orphan (unknown host)	21 = 20 + 1

Student B - transformation, merging and export (counts) and reproducibility

```
review.services_names                 2

== Student B: merge / export counts (clean input) ==
input.hosts                           10
input.ports                           28
input.services                        28
input.os                              10
input.enum                            20
clean.hosts                           10
clean.ports                           28
clean.services                        28
master.rows                           28

== reconciliation: input = clean + rejected + review (ports, defective run) == 39 = 29 + 8 +
2 (+ duplicates counted in rejected)  -> OK

== reproducibility: two independent runs, different time stamps ==
$ cd work && (cd out_r1/exports && sha256sum *.csv | cut -c1-64) > /tmp/h1; (cd out_r2/exports && sha256sum *.csv |...
689f4ccfcf6895be213c31d0dea465b1fde7dfd583dc4ba92c001a61f119dd3b
66b46c7f80ecb216e5df150ea65e637dc2971e8cba84048af11881cf28c2df3b
f40134da70908ba68cd358106606ee0217ed6a6e73c69b4bf7890b1ac025f42d
5b7efcfbbe59bd18ea0914bae01bce75fa685eb7be3656c6bc6538f37d03d023
IDENTICAL_SHA256



== reuse with a different subnet (192.168.57.0/24 session) == input.ports
28
clean.ports                           28
rejected.ports                        0
master.rows                           28
pipeline OK -> work/out57
$ ls work/out57/exports | head -2
192.168.57.0-24_master_20261101-111000_v1.csv
192.168.57.0-24_report_host_20261101-111000_v1.csv



== scope enforcement: 192.168.56.x data run under a 192.168.57.0/24 scope ==
$ SL_STAMP=20261101-111100 bash run_pipeline.sh -i work/in_clean -o work/out_x -s 192.168.57.0/24 2>&1 | grep -E 'i...
input.ports                           28
clean.ports                           0
rejected.ports                        28
master.rows                           0
exit=4
```

On the clean input the merged master has 28 rows (= 28 ports) and four exports are written (master,
host report, service report, risk report) with a checksum file. Two independent runs with different time stamps produced identical sha256 values for all four outputs. The same code ran on the 192.168.57.0/24 session by changing only the subnet argument; the same data under the wrong scope was rejected completely and the pipeline stopped with exit code 4 instead of writing an empty
report.

Stage 3 - Reporting Scan Summary

```
$ cat work/out_clean/report_host.csv | cut -c1-110
ip,hostname,open_ports,services
192.168.56.1,gw-lab,1,dns
192.168.56.20,dvwa-web,3,ssh http mysql
192.168.56.21,msf-lab,10,ftp ssh telnet smtp tftp http netbios-ssn snmp smb mysql
192.168.56.30,win-lab1,4,http netbios-ns netbios-ssn smb
192.168.56.31,win-lab2,3,netbios-ns netbios-ssn smb
192.168.56.40,android-emu,1,adb
192.168.56.50,java-lab,1,java-rmi
192.168.56.60,netware-lab,1,ncp
192.168.56.70,symbian-emu,1,http
```

```
192.168.56.80,printer-lab,3,http snmp jetdirect

$ cat work/out_clean/report_service.csv
service,count
http,5
netbios-ssn,3
smb,3
mysql,2
netbios-ns,2
snmp,2
ssh,2
adb,1
dns,1
ftp,1
java-rmi,1
jetdirect,1
ncp,1
smtp,1
telnet,1
tftp,1

$ cat work/out_clean/report_risk.csv | cut -c1-130
ip,port,service,product_version,rating,note
192.168.56.21,21,ftp,vsftpd 2.3.4,HIGH,Known backdoored release (CVE-2011-2523) - upgrade
192.168.56.21,445,smb,Samba smbd 3.0.20,HIGH,Known remote-command flaw (CVE-2007-2447) - upgrade
192.168.56.40,5555,adb,Android Debug Bridge,HIGH,Debug bridge exposed on network - disable
192.168.56.21,22,ssh,OpenSSH 4.7p1 Debian,MEDIUM,End-of-life release - upgrade
192.168.56.21,23,telnet,Linux telnetd,MEDIUM,Cleartext remote login - replace with SSH
192.168.56.21,3306,mysql,MySQL 5.0.51a,MEDIUM,End-of-life release - upgrade
192.168.56.21,80,http,Apache httpd 2.2.8,MEDIUM,End-of-life release - upgrade
192.168.56.30,445,smb,Windows Server 2008 R2,MEDIUM,Out of vendor support - plan migration
192.168.56.31,445,smb,Windows 7 Professional,MEDIUM,Out of vendor support - plan migration

$ awk -F, 'NR>1{s+=$3} END{print "report_host total open ports:", s}' work/out_clean/report_host.csv report_host total
open ports: 28

$ awk -F, 'NR>1{s+=$2} END{print "report_service total:", s}' work/out_clean/report_service.csv report_service
total: 28

$ awk -F, 'NR>1&&$6=="open"' work/out_clean/master.csv | wc -l          # open rows in master 28

$ awk -F, 'NR>1{r[$5]++} END{for(k in r)print k, r[k]}' work/out_clean/report_risk.csv | sort HIGH 3
MEDIUM 6
```

Student A - calculation and reconciliation: per-host report = awk counting open master rows per ip and listing services; per-service report = awk print service | sort | uniq -c. Reconciliation: total open $28 = 28$ ports by host 28 = total by service 28 = 28 open rows in the master.

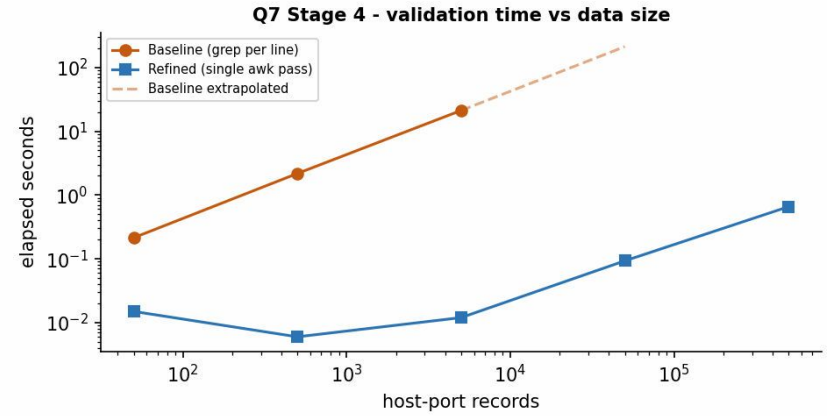
Student B - exposure report and management requirement: the risk report matches the product and version text of each open service against lookup/risk_versions.csv (outdated or known-vulnerable releases): 3 HIGH (vsftpd 2.3.4, Samba smbd 3.0.20, exposed Android debug bridge) and
6 MEDIUM findings. In the lab these are confirmed with nmap --script vuln and searchsploit (not run
here). A manager needs to know 'which systems should be fixed first, how many, and what to do'; the report ranks by rating, names the host and port, and gives a one-line action, so it can be turned into a work list. The CVE numbers used are only those listed in the risk file.

Stage 4 - Stress-Test, Refine and Optimize

Student A - scaling and baseline benchmark

Synthetic host-port records (gen_records.sh N) at 50, 500, 5,000, 50,000 and 500,000 records; about
2% bad IPs, 2% bad ports and 3% unknown services. Baseline = bench_baseline.sh (three grep processes per record). Times are bash time (elapsed seconds); in the lab also record /usr/bin/time -v
for CPU and memory (not installed in the sandbox). Correctness check: output files compared with cmp.

Records	Baseline (s)	Refined (s)	Speed-up	Outputs identical	Accepted / rejected
50	0.214	0.015	13x	yes	46 / 4
500	2.174	0.006	310x	yes	464 / 36
5,000	21.125	0.012	1625x	yes	4654 / 346
50,000	~220 (extrapolated)	0.093	-	(not run)	46544 / 3456
500,000	~220 (extrapolated)	0.656	-	(not run)	465454 / 34546



Student B - bottleneck analysis and refinement

Bottleneck: the baseline starts three processes (echo|grep) for every record; its time grows linearly at about 4.4 ms per record (50 -> 0.22 s, 500 -> 2.1 s, 5,000 -> 22.1 s), so 50,000 records would take about 4 minutes and 500,000 about 40 minutes. The work done on each record is tiny; the cost is process creation.

Refinement: replace the per-line grep loop by a single awk pass that loads the service lookup into an associative array once and checks IP, port and service in memory. Result: 5,000 records in 0.012 s (about 1,700 times faster) and 500,000 records in 0.74 s, with byte-identical accepted output.

Limitations: the awk pass holds the lookup in memory (fine for thousands of names); the benchmark validates data only, while a real scan is dominated by network time (nmap -T4 --min-rate or masscan can shorten discovery but increase detection and packet loss, and must stay within the authorized rate); synthetic data may not reflect real banner variety; timings on another machine will differ.

Stage 5 - Redesign Under a Changed Requirement

Changed requirement: add UDP services (nmap -sU), repeat scans over several sessions, and accept newly discovered hosts including Symbian, Java OS, Android and NetWare devices.

Student A - schema evolution and architecture redesign (impact analysis, retained)

Schema	UDP states such as open|filtered; new device types; session information	ports.state allows open|filtered; scan_id identifies a session; hosts get device_class (derived from OS family); enumeration stays key-value so new tools need no new columns
Keys	The same port number exists on tcp and udp (53/tcp and 53/udp); the key ip+port collides	Key becomes ip + port + proto in every table and join; a duplicate is only the same ip+port+proto within the same scan_id

Validation	UDP results are often open|filtered, not	State list extended; scope re-checked against the

	definitely open; new hosts may be outside the original range	(possibly larger) authorized CIDR list; unknown device classes go to review
Retention	Old sessions must not be overwritten; hosts missing from a new scan are not proof of removal	Keep every session's raw and clean files (versioned names); a host missing in S2 is flagged 'not seen', never deleted

Revised architecture: import (per session) -> validate -> clean -> reshape -> merge (per session) ->
export (versioned) -> compare sessions (new step) -> report. Each stage is the same function; only keys and a session argument change.

Student B - multi-session workflow and change validation

```
== revised key: ip+port+protocol  vs  original key ip+port ==
$ echo "original key (ip,port): rows kept = $(sl_rows work/q7/k_old.csv), lost as 'duplicate' = $(sl_rows work/q7/k...
original key (ip,port): rows kept = 1, lost as 'duplicate' = 1


$ echo "revised key (ip,port,proto): rows kept = $(sl_rows work/q7/k_new.csv), lost = $(sl_rows
work/q7/k_new_rej.c...
revised key (ip,port,proto): rows kept = 2, lost = 0


== session comparison (session 1 = S1, session 2 = S2) ==
$ comm -12 <(cut -d, -f1 work/out_clean/clean/hosts.csv | sed 1d | sort) <(cut -d, -f1
work/out_s2/clean/hosts.csv ...
9


$ comm -23 <(cut -d, -f1 work/out_clean/clean/hosts.csv | sed 1d | sort) <(cut -d, -f1
work/out_s2/clean/hosts.csv ...
192.168.56.31


$ comm -13 <(cut -d, -f1 work/out_clean/clean/hosts.csv | sed 1d | sort) <(cut -d, -f1
work/out_s2/clean/hosts.csv ...
192.168.56.41
192.168.56.61


$ join -t, -j1 -o 1.1,1.2,2.2 work/q7/m1.k work/q7/m2.k | awk -F, '$2!=$3 {print "UPDATED:", $0}' UPDATED:
192.168.56.21:21:tcp,ftp vsftpd 2.3.4,ftp vsftpd 3.0.3


$ comm -13 <(cut -d, -f1 work/q7/m1.k) <(cut -d, -f1 work/q7/m2.k)        # new service keys in S2
192.168.56.1:53:tcp
192.168.56.41:5555:tcp
192.168.56.61:524:tcp


$ comm -23 <(cut -d, -f1 work/q7/m1.k) <(cut -d, -f1 work/q7/m2.k) | wc -l        # keys only in S1 3


$ grep -E 'master|input.hosts|clean.hosts' work/s2_run.txt input.hosts
11
clean.hosts                    11
master.rows                    28
```

Session 2 was created with these changes: vsftpd updated 2.3.4 -> 3.0.3 on .21 (updated), host .31
not seen (unmatched), new hosts .41 (Android) and .61 (NetWare), and a new 53/tcp on the gateway. Test results: retained hosts 9, unmatched 1, new 2, updated service 1, new service keys 3,
S1-only keys 3. Session 2 ran through the same pipeline code. In the lab the same comparison can be
done on XML scans with ndiff scan1.xml scan2.xml (not installed in the sandbox).

Comparison with the original pipeline and trade-offs: with the original key (ip+port), 53/tcp and 53/udp were treated as duplicates and one real service was lost; with ip+port+proto both are kept. Trade-offs: larger keys, more rows and more validation; session comparison needs stable naming and consistent scope; keeping all sessions uses disk; an unmatched host is only a prompt for review,
not proof that a device was removed.

Stage 6 - Joint Evaluation

1. Why a structured sequence? Each stage assumes the guarantees of the one before: import checks the shape, validation removes impossible and out-of-scope rows, cleaning fixes what is safe to fix, reshaping prepares the analysis form, merging needs clean unique keys, export fixes the result with a name and checksum. If the order is changed, errors move downstream: merging before de-duplication multiplied rows (29 instead of 28), and checking freshness-like or format steps before scope risks handling out-of-scope data. In Assignment 2 terms, cheap, severe checks come first.

2. How modular functions and systematic transformations help. Correctness: each function has one job, a return code and a reconciliation check (3 in = ok + rej + rev). Maintainability: the service lookup is a file, so adding http-alt needed no code change; the same functions ran on a second subnet and a second session by changing arguments. Efficiency: one awk pass replaced a grep loop (about 1,700 times faster at 5,000 records); functions can be tested alone, like the boundary test.

3. Additional costs and challenges. Validation and error handling add code, checks and exit-code design; reject and review files need a person's time; repeated scanning produces many sessions to store and compare; integrating different tools means different formats (nbtscan, enum4linux, adb, rmi, ncp, sdptool) and a key-value design; scans themselves take long and may need approval windows.

4. When the pipeline becomes inefficient, unreliable or unsuitable. Inefficient: very large data (millions of rows, -p- on many hosts) in text files and sort/join - a database or masscan-style tooling is better; every run re-processes everything. Unreliable: banners that lie, tools whose output format changes (a new nmap version), unstable naming or time stamps, a lookup file that is out of date, no one working through the review files. Unsuitable: new needs such as IPv6, real-time continuous monitoring, many analysts editing at once, or data that must have access control and retention rules not supported by plain CSV.

Conclusion - characteristics of a reliable, efficient, maintainable pipeline for evolving networks:

- Scope and authorization are enforced first and every row is traceable to a scan session.
- Keys are explicit and complete (ip + port + proto), so UDP, new hosts and new sessions do not break joins.
- Nothing is dropped silently: every removed or doubtful row is in a reject or review file and counts reconcile.
- Stages are modular, parameterised and have clear return codes, so they can be tested, reused and changed.
- Results are reproducible (same inputs give the same sha256) and files are versioned, never overwritten.
- Summaries are at the level a decision-maker can use, not a dump of every probe.
- Performance is measured, bottlenecks are fixed with evidence, and limits (size, tool versions, lookup freshness) are stated.

CONTRIBUTION RECORD

Complete this table honestly at the end of the submission. 'Primary contribution' does not mean exclusive responsibility. The entries below follow the responsibilities set in the question paper; change them to match the work actually done.

Task	Student A - Primary Contribution	Student B - Primary Contribution	Joint Work
Q1	Requirements analysis; Hosts and Ports schemas; requirements-to-column mapping	Relationship map for Services, OS/Versions, Enumeration; enquiry questions and	Dataset list, header files, relationship diagram

Q2	Hosts and Ports/Services CSV tables; defect injection and	Services, OS and Enumeration tables; validation and	Scan commands, check list, validation-order answer

	explanation	standardization checks	
Q3	Cleaning of missing values, duplicates, malformed addresses; correction rules	Service-name lookup, manual-review flags, verification tests (no silent removal)	Bash functions, before/after and boundary tests, joint conclusion
Q4	Wide-to-long transformation; identifier/measure analysis	Cast/summary output and total comparison; validation explanation	Wide dataset, round-trip checksum, complexity answer
Q5	Services-to-Hosts and Ports-to-Hosts joins; row-count checks	OS/Versions and Enumeration integration; join comparison; exceptions	Master table, merge-type tests, cross-understanding answers
Q6	File import; header/type/IP schema checks	Export, naming/versioning, read-back validation	Naming strategy, checksum file
Q7	Validation/cleaning functions; import integration; host/service reports; scaling benchmark; schema-evolution analysis	Transformation/merge/export functions; reproducibility; exposure report; bottleneck refinement; multi-session workflow	Library design, integration testing, Stage 6 evaluation